

MimicAgent: Quadruped Skills via Prompt-to-Trajectory Generation

Lucky Kant Nayak*, Narayanan Palghat Parameswaran*, Neehar Peri, Deva Ramanan
Carnegie Mellon University

Abstract— We present MimicAgent, a prompt-to-trajectory generation framework for learning dynamic quadruped skills. Although reward shaping is extensively used when training quadruped policies, navigating the resulting reward landscape is notoriously difficult, requiring hours of “graduate student descent”. Eureka attempts to automate reward design with LLMs, but we find that it struggles to generalize across diverse skills and morphologies. Our key observation is that it is far easier for a human – and by association, an LLM – to generate reference motions than to shape reward functions. Our hypothesis is motivated by the success of example-guided RL for humanoids, which exploits large-scale motion capture datasets as references for training locomotion policies. Unlike humanoids, quadrupeds lack such reference motion data. Towards this end, we propose MimicAgent, an agentic harness that, given a skill prompt, generates quadruped reference trajectories with coding agents. These coarse reference trajectories are then used to train example-guided RL policies that are deployable in simulation and in the real-world. Notably, we find that when prompting Claude Fable 5.1 within our agentic harness, 87% of prompts yield semantically aligned reference trajectories. Please see our project page for accompanying videos.

I. INTRODUCTION

In recent years, learning-based locomotion policies have demonstrated impressive agility and robustness across a wide range of behaviors [1] and terrains [2]. However, training such RL policies typically relies on carefully tuned rewards. For complex and highly dynamic skills, designing bespoke reward functions is notoriously difficult and often requires extensive trial-and-error [3]. This challenge has motivated recent efforts to eliminate manual reward engineering entirely.

LLMs for Reward Design. Recent work has explored using LLMs to automate reward design [4], [5], [6], [7]. For example, Eureka [5] leverages coding LLMs to generate and iteratively refine reward functions from natural language task descriptions. While promising, these methods still depend on hand-engineered success metrics to evaluate candidate rewards and often struggle to generalize across diverse skills and morphologies. In practice, generated reward functions are brittle and often don’t accurately follow natural language guidance, particularly for underexplored robot embodiments such as wheeled quadrupeds (Table I).

Learning from Examples. An alternative to reward-centric RL is learning from demonstrations. Example-guided RL has been a key enabler for recent progress in humanoid locomotion, where motion capture datasets [8], [9], [10], [11], [12], or manually annotated keyframes [13], [14] provide rich supervision. However, quadrupeds lack such large-scale, publicly available datasets. Recent works [15] curate libraries of simple quadruped motion patterns like walking, trotting,

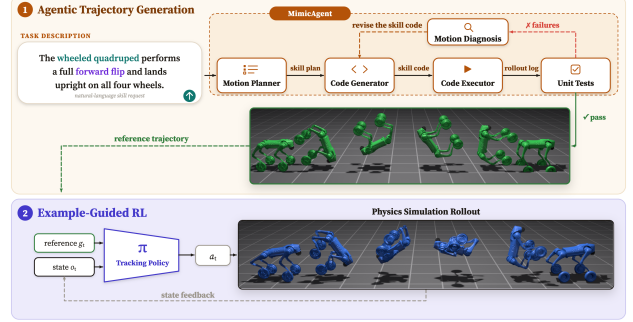


Fig. 1: **Overview.** MimicAgent uses LLM coding agents within a recursive self-improvement loop to generate coarse reference trajectories (shown in green) from text prompts. Our agentic pipeline converts short text prompts into detailed skill descriptions, translates these into executable functions that describe base and foot motion over time, and outputs them as reference trajectories. We show that these coarse motions are effective reference targets for example-guided RL (with policy rollouts shown in blue).

and galloping. However, such efforts are orders of magnitude smaller and less diverse than comparable humanoid datasets. This data scarcity limits the applicability of demonstration-based methods despite their strong empirical performance.

LLMs for Trajectory Generation. To address this gap, we propose MimicAgent, an LLM-based agentic pipeline that directly generates text-guided example trajectories for policy learning. *Our key observation is that it is far easier for a human – and by association, an LLM – to generate reference motions than to shape reward functions.* Given a natural language skill description, MimicAgent produces executable code defining base motion and foot trajectories. This code is then executed in a MuJoCo engine using forward kinematics to produce kinematically feasible trajectories, without running a full physics simulator. The resulting trajectories are consumed by a downstream imitation-based RL pipeline like DeepMimic [13].

Contributions. We present three major contributions.

- 1) *MimicAgent Framework.* We introduce a prompt-to-trajectory agentic harness specifically designed for learning dynamic quadruped skills.
- 2) *Learning from Coarse References.* We demonstrate that coarse trajectories generated by MimicAgent are sufficient for training successful sim-to-real policies.
- 3) *Trajectory Synthesis vs. Reward Shaping.* Our experiments highlight the limitations of prior LLM-based approaches that rely on reward shaping, and demonstrate the effectiveness of LLM-based trajectory synthesis.

II. RELATED WORKS

Quadrupedal Locomotion has matured significantly in recent years [16], [17]. Early work primarily focused on training legged robots to execute complex contact sequences to achieve different locomotion gaits. A common approach is to use model predictive control (MPC) to track predefined contact patterns [18], enabling canonical gaits such as trotting [19], pacing [20], bounding [21], and galloping [22]. Despite their success, MPC-based approaches require hand-designed reference trajectories and incur high computational costs, limiting behavioral diversity. Recent work extends these ideas to agile bipedal quadrupedal motions [23] and whole-body loco-manipulation [24]. In contrast, our approach leverages example-guided reinforcement learning, which can be more intuitive than designing foot contact sequences.

Text-to-Robot Control is an active area of research for both quadrupeds [25], [26] and humanoids [27], [6]. Early methods relied on structured text templates or NLP tools like parse trees to extract constraints and generate robot trajectories [28]. Other approaches used representation learning to train language-conditioned policies that map free-form instructions directly to actions [29], [30]. However, such approaches require large annotated datasets with paired text and robot control, which is difficult to collect for diverse locomotion behaviors. More recent methods prompt LLMs to generate robot code, bridging the gap between language and motor control via intermediate representations like high-level plans, primitive skills, or trajectories [31], [32], [33]. Inspired by prior work, we leverage LLMs to directly generate reference trajectories for imitation learning.

III. DYNAMIC SKILLS VIA TRAJECTORY SYNTHESIS

MimicAgent aims to automate motion imitation learning by replacing manual reward engineering with an agentic motion synthesis pipeline. Rather than navigating a complex and brittle reward landscape, MimicAgent leverages the observation that generating a plausible reference trajectory is often significantly easier than designing a robust reward function for the same behavior.

Agentic Motion Synthesis Pipeline. MimicAgent is an iterative pipeline that decomposes motion synthesis into *motion planning*, *code generation*, *execution*, and *refinement* (Figure 2). The input to the system is a natural language skill prompt, which is processed by a *motion planner* agent to produce a detailed plan encoding the temporal and kinematic structure of the skill. Each motion plan is a textual description of the overall motion intent and stability strategy, the decomposition into sub-phases with associated leg contact states, coordination patterns across leg groups, base linear and angular velocity commands in the body frame, and qualitative leg trajectories relative to the body. It also provides handoff notes for the code generation agent for controller implementation.

Each skill is parameterized by a normalized phase variable $\phi \in [0, 1]$, mapping motions of arbitrary duration into a unified phase-indexed form. A reference trajectory is defined as a sequence of keyframes sampled at discrete phase steps

$\phi_t = t/T$, denoted as $\tau = \{s_0, s_1, \dots, s_T\}$. At each phase step t , the keyframe state is: $s_t = [\mathbf{p}_t^{\text{base}}, \boldsymbol{\alpha}_t^{\text{base}}, \boldsymbol{\theta}_t]$, where $\mathbf{p}_t^{\text{base}} \in \mathbb{R}^3$ represents the robot base 3D position, $\boldsymbol{\alpha}_t^{\text{base}} \in \mathbb{R}^4$ is the base orientation represented as a unit quaternion, and $\boldsymbol{\theta}_t \in \mathbb{R}^n$ is the vector of n joint rotations. This representation enables a compact and interpretable parameterization capable of expressing a wide range of locomotion behaviors.

Code Generation and Trajectory Execution. Given the motion plan, a *code generation* agent synthesizes an executable program that defines the temporal functions and sequence for base and foot trajectories. This generated code is executed via forward kinematics within a MuJoCo simulator. Importantly, we do not perform any physics simulation. As a result, the synthesized trajectories are kinematically consistent but generally violate dynamic constraints. MimicAgent explicitly embraces this tradeoff, delegating dynamic feasibility to the reinforcement learning stage.

Task-Agnostic Unit Tests. To validate generated trajectories without introducing task-specific biases, MimicAgent employs a set of *task-agnostic unit tests*. Rather than optimizing task rewards, MimicAgent filters generated motions using task-agnostic constraints, enabling scalable reference trajectory generation without manually specifying task-specific success criteria. These tests are designed to reject physically implausible or degenerate trajectories while remaining broadly applicable across skills. Each unit test computes a scalar reward, and the aggregate score determines whether a trajectory passes validation. The unit tests include:

- **Base Height Envelope.** The base height is constrained to remain within a reasonable envelope relative to the ground. Heights below 0.1 meters or those exceeding twice the standing base height are penalized. This test returns a binary reward of +1 or -1.
- **Joint Violation Penalty.** Joint limit violations are penalized proportional to their frequency. The reward is computed as $r_{\text{joint}} = -1 \times N_{\text{violated}}$, where N_{violated} denotes the number of joints exceeding their limits.
- **Ground Penetration.** Foot-ground penetration is penalized based on severity. Trajectories in which all four feet penetrate the ground incur a large penalty, while partial penetration results in a smaller penalty. The reward is bounded between +1 and -1.
- **Feet Air-Time Constraint.** For non-aerial skills, at least one foot must remain in contact with the ground for half of the motion cycle. This constraint prevents degenerate motions that leave the ground for extended durations.

Importantly, these unit tests are skill-agnostic and do not encode task success, performance objectives, or stylistic preferences. They serve as structural validity checks, ensuring that generated trajectories are suitable as reference trajectories. In contrast, Eureka [5] relies heavily on carefully designed task-specific success functions to generate reward signals. Defining such success criteria is a time-intensive process that prohibits learning large skill libraries.

Recursive Self Improvement Loop. If a trajectory fails validation, a diagnostic agent analyzes the failure in the context of the full system state, including the skill descrip-

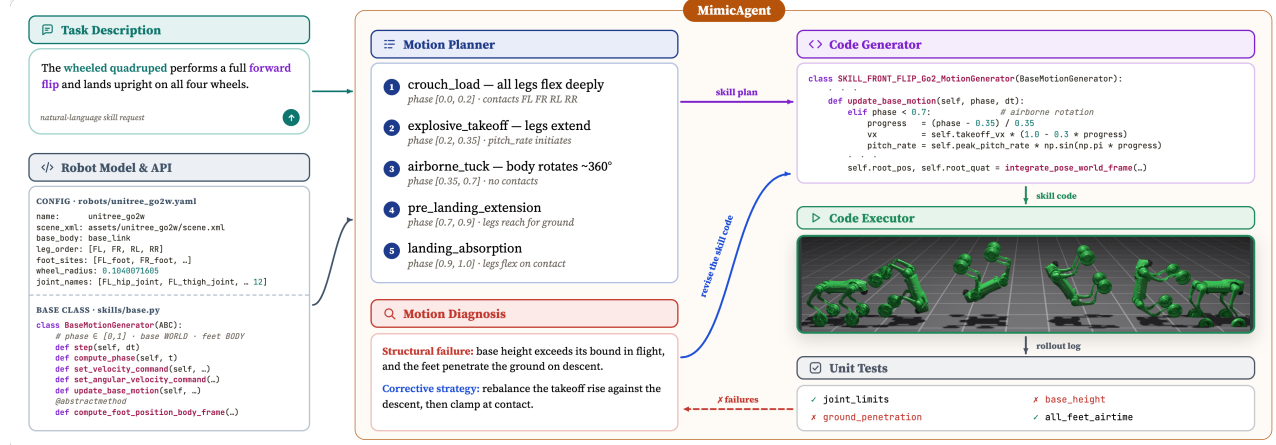


Fig. 2: **MimicAgent Pipeline.** Given the prompt “the wheeled quadruped performs a full forward flip and lands upright on all four wheels”, our motion planner agent draws on prior knowledge of the robot model and the simulation environment API to decompose the skill into subparts. The code generator agent converts this phase plan into executable base and foot trajectories, which the kinematic executor renders. Task-agnostic unit tests then check the rendered motion for violations. The motion-diagnosis agent summarizes any failed checks and returns them as targeted feedback for code revision. This self-improvement loop repeats until the trajectory passes validation or the iteration budget is exhausted.

tion, motion plan, generated code, and unit test outcomes. The diagnostic agent produces natural language feedback identifying likely failure modes. A code modification agent then revises the trajectory-generating program accordingly. This process iterates until a trajectory passes all unit tests or a fixed iteration budget is reached ($N=3$). Valid trajectories are returned for downstream imitation learning.

By shifting the focus from reward optimization to reference trajectory synthesis, MimicAgent avoids the brittle and time-consuming process of manual reward engineering. Unlike LLM-based reward design methods that require repeated policy training and hand-crafted evaluation criteria, MimicAgent generates supervision directly in trajectory space. This makes the framework efficient, interpretable, and naturally extensible to diverse quadruped skills.

Example-Guided RL and Sim-to-Real Transfer. Given a coarse reference trajectory, we formulate motion imitation as goal-conditioned reinforcement learning, following DeepMimic [13]. Similar to HOVER [11], we first train a privileged teacher policy with PPO using full simulator state and subsequently distill it into a deployable student policy whose observations can be reconstructed from onboard sensing. At each control step t (50 Hz), the policy $\pi(a_t | s_t^p, s_t^g)$ maps the proprioceptive observation s_t^p and reference-goal observation s_t^g to a normalized action vector $a_t \in [-1, 1]^{16}$ for the 16-DoF Go2-W (12 leg joints and 4 wheels). The teacher uses global yaw information available only in simulation, whereas the student employs a yaw-invariant observation representation suitable for hardware deployment. Their self-orientation and reference-orientation inputs are defined as follows:

$$s_t^{p\text{-teacher}} = [\mathbf{q}_t^{\text{root}}, \boldsymbol{\omega}_t^{\text{root}}, \boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, a_{t-1}], \quad (1)$$

$$s_t^{p\text{-student}} = [\mathbf{g}_t, \boldsymbol{\omega}_t^{\text{root}}, \boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, a_{t-1}], \quad (2)$$

and the goal blocks are

$$s_t^{g\text{-teacher}} = [\Delta \mathbf{p}_t^{\text{root}}, \hat{\mathbf{q}}_t^{\text{root}}, \hat{\mathbf{v}}_t, \hat{\boldsymbol{\omega}}_t, \hat{\boldsymbol{\theta}}_t, \hat{\boldsymbol{\theta}}_t, \Delta \mathbf{f}_t, \phi_t], \quad (3)$$

$$s_t^{g\text{-student}} = [\hat{\mathbf{g}}_t, \hat{\mathbf{v}}_t, \hat{\boldsymbol{\omega}}_t, \hat{\boldsymbol{\theta}}_t, \hat{\boldsymbol{\theta}}_t, \Delta \mathbf{f}_t, \phi_t], \quad (4)$$

where $\mathbf{q}_t^{\text{root}}$ is the root orientation quaternion (sim-only), $\mathbf{g}_t$ is the body-frame gravity direction (projected gravity), $\boldsymbol{\omega}_t^{\text{root}}$ is the base angular velocity, $\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t$ are the joint positions and velocities, a_{t-1} is the previous action, $\Delta \mathbf{p}_t^{\text{root}}$ and $\Delta \mathbf{f}_t$ are the body-frame root-position and end-effector tracking errors, $\hat{\mathbf{v}}_t, \hat{\boldsymbol{\omega}}_t$ are the reference velocities, and $\phi_t \in [0, 1]$ is the cyclic motion phase. Swapping the teacher’s absolute-orientation channels for the single yaw-free reference gravity $\hat{\mathbf{g}}_t$ makes the student’s observation viable on hardware and free of IMU yaw drift.

The teacher also receives a privileged block s_t^{priv} of ground-truth base velocity and height, link positions, contact flags, a short future-reference lookahead, and the per-environment domain-randomization parameters. The teacher network is a symmetric actor-critic, where both networks consume $[s_t^{p\text{-teacher}}, s_t^{g\text{-teacher}}, s_t^{\text{priv}}]$. In contrast, the student actor only sees the deployable channels plus a 50-step proprioceptive history H , while its critic reuses s_t^{priv} .

Once the teacher network converges, it supervises the student via DAgger [34]. The student rolls out $\pi_{\text{student}}(a_t | s_t^{p\text{-student}}, s_t^{g\text{-student}})$, and the oracle $\pi_{\text{teacher}}(\hat{a}_t | s_t^{p\text{-teacher}}, s_t^{g\text{-teacher}})$ is queried on the matched privileged state for target actions $\hat{a}_t$. We behavior-clone ($\|a_t - \hat{a}_t\|^2$) on the deployable observation. Both stages use domain randomization and observation noise for robust sim-to-real transfer.

The final reward sums imitation terms, each an exponential tracking kernel $\exp(-k e)$, and hardware-aware regularization terms:

$$r_t = \underbrace{\sum_{m \in \mathcal{I}} w_m \exp(-k_m e_t^m)}_{\text{imitation}} + \underbrace{\sum_{n \in \mathcal{R}} w_n c_t^n}_{\text{regularization}}, \quad (5)$$

where e_t^m is the squared deviation between simulated and reference quantity for $m \in \{\text{joint position, joint velocity, root position, root rotation, root linear velocity, root angular velocity, end-effector}\}$. The regularization terms c_t^n penalize action rate, joint velocity, joint acceleration, hinge violations of the joint-position, velocity, and torque limits.

IV. EXPERIMENTS

We first describe our evaluation metrics and benchmark our baselines. We then ablate the quality of generated reference motions, comparing human-annotated keyframes against trajectories produced by our agentic synthesis pipeline, and measure the annotation time each requires. Further, we compare the policies learned by our model against hand-tuned RL expert policies for trotting and bounding. Lastly, we ablate the impact of unit tests and iterative refinement on our generated references.

Metrics. We quantitatively compare manual keyframing, Eureka [5], and MimicAgent with a 57 person user study conducted through Cint. Participants were presented with pairs of textual skill prompts and videos of trained policies corresponding to that task and were asked to rate how well the video of policy rollouts follows the prompt on a 5-point Likert scale. Videos were presented in a randomized order to eliminate ordering bias and participants were not informed which method was used to generate each video. In addition, we compute the mean squared error between the reference trajectories and the trajectories produced by the learned policy for both human annotated keyframes and MimicAgent’s keyframes to measure how faithfully the learned policy follows the demonstration.

Baselines. We compare MimicAgent against Eureka [5] and DeepMimic [13] policies learned from human-annotated keyframes across seven skills: Go2 Trot, Go2 Bounding, Go2-W Front Flip (FF), Go2-W Side Flip (SF), Go2-W Aerial Crossover (AC), Go2-W Reverberating Yaw Pulse (RYP), and Go2-W Crab Diagonal Scuttle (CDS), described below:

- *UniTree Go2 Trot.* A symmetric diagonal gait alternating front-left and rear-right with front-right and rear-left, maintaining continuous ground contact and a stable torso while tracking commanded forward velocity.
- *UniTree Go2 Bounding.* A dynamic gait where the front legs push off together followed by the rear, producing a brief aerial phase and pronounced vertical oscillation under forward propulsion.
- *UniTree Go2-W Front Flip.* Strong forward pitch momentum carries the robot fully airborne through a 360 rotation about its lateral axis, landing upright with controlled impact.
- *UniTree Go2-W Side Flip.* A rapid roll about the longitudinal axis carries the robot fully airborne through a 360 side rotation, recovering to a stable stance.

- *UniTree Go2-W Aerial Crossover.* While carving forward on its wheels, the robot lifts alternating diagonal leg pairs off the ground, executing rapid mid-air crossover motions.
- *UniTree Go2-W Reverberating Yaw Pulse.* An oscillatory motion with a decreasing amplitude envelope that mimics physical damping. Phase controls the amplitude decay and pulse frequency, giving an accelerating–decelerating rhythm visually distinct from constant-rate spinning.
- *UniTree Go2-W Crab Diagonal Scuttle.* The robot scuttles diagonally with its body axis held perpendicular to the travel vector, decoupling orientation from travel direction. Phase coordinates cross-body leg sweeps in which front and rear act in opposition.

Eureka. Eureka formulates reward generation as evolutionary search over reward programs. Given a natural-language task description ℓ , environment source code $\mathcal{M}$, a policy optimization algorithm $\mathcal{A}$, and a task-level fitness function F , the LLM samples a population of K candidate reward programs $\{R_1, \dots, R_K\}$. Each reward program produces a scalar reward and a dictionary of interpretable reward components. For each candidate, Eureka trains a policy $\pi_k = \mathcal{A}(\mathcal{M}, R_k)$ and evaluates its performance using $s_k = F(\pi_k)$. At each iteration, the best-performing candidate $R^* = \arg \max_k s_k$ is retained and supplied to the LLM together with reward-reflection feedback. This feedback summarizes the task-level fitness and the values of individual reward components at intermediate policy-training checkpoints, enabling the LLM to modify their scales or composition. This constitutes one evolutionary iteration; we repeat for N iterations, optionally with random restarts, and return the highest-fitness program. We use the default configuration with $N = 5$ evolutionary-search iterations and $K = 16$ candidate rewards per iteration, yielding 80 candidate reward programs per skill.

Eureka’s search depends on how accurately F captures the target behavior. Since no predefined evaluation metrics exist for our novel Go2-W skills, we manually construct a task-specific fitness function for each skill. For example, the fitness function for a side flip measures whether the robot completes the desired airborne rotation and recovers to an upright configuration. This provides Eureka with explicit task-success supervision during reward search. In contrast, MimicAgent applies a shared set of structural validation checks across skills and does not optimize a task-specific fitness metric. Despite this favorable evaluation setting, we find that Eureka produces less reliable policies for the stylistically diverse Go2-W behaviors considered in our experiments.

Keyframing Reference Motion Tool. To streamline reference-motion animation, we built a custom keyframe annotation tool with Claude that visualizes the kinematics of a robot model in MuJoCo and lets users “puppeteer” the robot to generate reference trajectories. Our tool uses click-and-drag mechanics for coarse spatial adjustment, and a slider-based UI panel for fine-grained control of individual foot positions (X, Y, Z) and root body orientation (roll, pitch, yaw). Leg mirroring, a keyframe timeline, and trajectory format selection further accelerate annotation. We argue that this interactive

TABLE I: **Comparison to State-of-the-Art.** We conduct a user study (n=57) to compare the semantic alignment of policies generated with human-annotated keyframes, Eureka, and MimicAgent on a scale of 1 to 5. Higher is better.

Robot Method ↓ / Skill →	Go2		Go2-W				
	Trot	Bound	SF	FF	AC	CDS	RYP
Keyframe Tool	2.9 ± 1.4	3.5 ± 1.4	3.5 ± 1.2	4.2 ± 1.0	2.3 ± 1.3	2.5 ± 1.2	1.8 ± 1.2
Eureka	2.6 ± 1.5	2.8 ± 1.4	1.4 ± 1.1	1.1 ± 0.3	1.9 ± 1.1	2.1 ± 1.1	4.3 ± 1.4
MimicAgent	4.4 ± 1.0	3.2 ± 1.4	4.9 ± 0.3	4.6 ± 0.8	4.2 ± 1.0	4.5 ± 0.5	2.6 ± 1.4

TABLE II: **Comparison of Time-to-Annotation.** We evaluate the time to annotation (in minutes) between human-annotated keyframes and MimicAgent keyframes. Lower is better.

Robot Method ↓ / Skill →	Go2		Go2-W				
	Trot	Bound	SF	FF	AC	CDS	RYP
Keyframe Tool	2.6	4.5	4.5	4.4	7.6	14.5	12.4
MimicAgent	1.1	1.2	2.9	3.2	3.6	1.6	1.8

design lowers the barrier to authoring reference motion – for human operators and LLMs alike – relative to traditional reward shaping workflows.

Comparison with State-of-the-Art. We conduct a user study (n=57) comparing trained policies generated from human annotated keyframes, Eureka, and MimicAgent. Participants consistently prefer MimicAgent policies for Trotting (+1.8 vs. Eureka), Side Flip (+3.5 vs. Eureka), Front Flip (+3.5 vs. Eureka), Aerial Crossover (+2.3 vs. Eureka), and Crab Diagonal Scuttle (+2.4 vs. Eureka). For Bounding, users favor policies generated from human-annotated keyframes, though all approaches received relatively low ratings. Eureka achieves the highest score on Reverberating Yaw Pulse (+1.7 vs. MimicAgent). Notably, Eureka still relies on carefully specified task-specific success metrics and exhibits limited robustness when scaling to diverse, stylistically different skills. Therefore, we hand-engineer success criteria for Eureka’s policies to give it a better chance of succeeding. Despite this, performance degrades for many wheeled quadruped skills, likely due to the increased joint space complexity.

Time-to-Annotation. We compare the time required for generating human-annotated keyframes and MimicAgent generated keyframes in Table II. We find that MimicAgent consistently requires less time than manually annotated keyframes. The first five skills (trot, bounding, side flip, front flip, aerial crossover) are intentionally limited to relatively simple motions (e.g., unidirectional velocity or in-place yaw). In contrast, crab diagonal scuttle and reverberating yaw pulse represent more complex skills that incorporate asymmetric leg movements coupled with base rotations, which are time-consuming to keyframe manually. We posit that as skill complexity increases, human annotation time will grow significantly faster than MimicAgents skill generation time.

Comparison of Reference Motion Quality. We compare DeepMimic policies trained on reference trajectories with 150 frames generated by our manual keyframing tool and MimicAgent in Table III. Policies trained with MimicAgent reference trajectories consistently achieved lower tracking error across almost all skills and evaluation metrics, suggesting that the generated reference trajectories were smoother

TABLE III: **Comparison of Reference Motion Quality.** We evaluate the tracking error of policies trained with human-annotated and MimicAgent keyframes. Across all skills and metrics, MimicAgent policies achieve lower mean squared error on both pose-level errors (root and body position and rotation) and motion-level errors (linear and angular velocities). Lower is better.

Method	Robot Metric ↓ / Skill →	Go2		Go2-W				
		Trot	Bound	SF	FF	AC	CDS	RYP
Keyframe Tool	E_{RP}	0.170	0.076	0.199	0.099	0.020	0.044	0.041
	E_{RR}	0.177	0.208	0.238	0.208	0.025	0.201	0.142
	E_{JR}	0.360	0.320	0.484	0.507	0.396	0.033	0.025
	E_{JV}	4.313	3.868	8.380	15.183	6.823	6.171	4.721
	E_{LV}	0.251	0.214	0.548	0.287	0.035	0.261	0.126
	E_{AV}	0.918	0.517	1.883	1.126	0.233	0.797	1.172
MimicAgent	E_{RP}	0.052	0.049	0.077	0.089	0.013	0.035	0.013
	E_{RR}	0.046	0.114	0.207	0.160	0.024	0.032	0.089
	E_{JR}	0.109	0.224	0.382	0.528	0.373	0.009	0.018
	E_{JV}	0.646	1.580	5.299	12.871	4.761	1.609	2.348
	E_{LV}	0.093	0.136	0.295	0.264	0.044	0.124	0.047
	E_{AV}	0.176	0.353	1.869	1.697	0.211	0.264	0.455

than the human-annotated trajectories. We track errors for root position in the global frame (E_{RP} , m), root rotation (E_{RR} , rad), joint rotation (E_{JR} , rad), joint velocity (E_{JV} , rad/s), root linear velocity (E_{LV} , m/s), and root angular velocity (E_{AV} , rad/s). This improvement is most pronounced for highly dynamic skills such as side flips, front flips, and aerial crossovers, where policies trained on manually keyframed trajectories exhibit significantly larger velocity and angular velocity errors. In contrast, policies trained on MimicAgent references result in improved motion timing and smoother rotational dynamics during policy learning. Even for simpler skills such as trotting and bounding, MimicAgent yields lower global pose error, reflecting higher reference motion quality. These results indicate that improved reference trajectory quality directly reduces downstream imitation error, demonstrating the effectiveness of MimicAgent as a scalable alternative to manual keyframing for imitation-based RL.

Comparison with Expert RL Controller. To test whether MimicAgent policies reach the locomotion quality of hand-tuned expert controllers, we compare against Walk-These-Ways (WTW) [1] on two canonical gaits: trot and bound. However, since these two methods are not guaranteed to produce gaits at the same frequency, their trajectories are not directly comparable. We therefore adopt Waveform Morphology Analysis (WMA), normalizing each trajectory to a single gait cycle for frequency-invariant comparison (Table IV). On trot, we find that MimicAgent tracks a 1.0 m/s command at 0.9999 m/s, a $6\times$ lower MAE than WTW, while attaining higher periodic stability (0.958 vs. 0.865). Its stance duty cycle of 49.8% closely matches the canonical 50/50 trot definition. Cycle duration varies by only ± 0.2 frames against ± 0.5 for WTW, confirming a more metronomic gait. On bound, MimicAgent tracks the commanded velocity with 30% lower error and again achieves higher periodic stability (0.959 vs. 0.863). Notably, MimicAgent spontaneously discovers a more dynamic gait topology; it’s 39.4% stance duty cycle (vs. 49.8%) suggests a genuine aerial flight phase that WTW does not exhibit.

Compute and Token Analysis. Tab. V compares the compute and token usage of Eureka and MimicAgent when

TABLE IV: **Comparison with Walk-These-Ways.** We compare the gait quality of an RL policy learned from MimicAgent’s reference with an expert-tuned controller at a commanded velocity of 1 m/s. MimicAgent achieves 6× lower velocity error on trot and 30% lower on bound.

Gait	Metric	MimicAgent (Ours)	WTW (Expert)
Trot	Mean Vel. (m/s)	0.9999	1.1959
	Vel. Error (MAE)	0.0351	0.1959
	Periodic Stab.	0.9575	0.8646
	Stance Duty (%)	49.8	47.0
	Cycle Dur. (Frames)	10.0 ± 0.2	16.6 ± 0.5
Bound	Mean Vel. (m/s)	1.0053	1.1930
	Vel. Error (MAE)	0.1495	0.2139
	Periodic Stab.	0.9587	0.8625
	Stance Duty (%)	39.4	47.7
	Cycle Dur. (Frames)	9.8 ± 0.8	16.7 ± 0.5

TABLE V: **Compute and Token Comparison.** We compare the compute and token usage between MimicAgent and Eureka below using 8× NVIDIA RTX 3090 GPUs. We average results over 7 skills, and compute token usage with Claude Sonnet 4.5.

Metric	Eureka	MimicAgent
Parallel Envs.	4,096	32,768
PPO Iterations	3,000	10,000
Avg. Wall Clock Time	5 h 45 min	6 h 22 min
Avg. GPU Hours	46.0	50.9
Avg. LLM Tokens	727K	17K

training the privileged-teacher network. Eureka evaluates 80 reward candidates per skill across 5 evolutionary search iterations, whereas MimicAgent trains a reference-tracking teacher under domain randomization for sim-to-real robustness. We exclude student distillation from this comparison, as it is a separate deployment-specific stage and Eureka’s policies are not deployable as trained. We find that the two methods use comparable training compute (46.0 vs. 50.9 GPU-hours), while Eureka consumes roughly 42× more LLM tokens for reward generation and refinement. However, videos on our project page show that Eureka learns stable locomotion skills yet rarely acquires aerial or parkour skills. Increasing its PPO budget would optimize the selected reward more thoroughly, but cannot correct an objective that fails to encode the intended behavior. We therefore argue that reward-candidate discovery, rather than training or token budget, is Eureka’s primary bottleneck on our evaluated skills.

Automatic Skill Generation. In addition to manually prompting MimicAgent, we can leverage LLMs to automatically generate large-scale skill libraries. We employ a dedicated skill proposal agent that is instructed to propose behaviors that differ in contact patterns, base motion (translation, rotation, or vertical motion), limb coordination, and stylistic or agility characteristics. To prevent duplication, we append all previously generated skills to the prompt context on subsequent iterations, repeating until we reach a target number of unique skills. MimicAgent then processes each description independently. As shown in Fig. 3, we generated

TABLE VI: **Direct Prompting vs MimicAgent Harness.** We compare unit test success rates between direct code generation and the MimicAgent critic-repair pipeline on 65 LLM proposed skills. Loop i reports cumulative success after i rounds of critic-driven self-repair. SA reports the human verified semantic alignment of generated reference trajectory with the skill prompt.

Model	Method	No Loop	Loop 1	Loop 2	Loop 3	SA
Sonnet 5	Direct	54%	—	—	—	49%
	MimicAgent	54%	74%	78%	79%	65%
Opus 5	Direct	49%	—	—	—	36%
	MimicAgent	68%	80%	91%	93%	82%
Fable 5.1	Direct	71%	—	—	—	68%
	MimicAgent	73%	91%	94%	95%	87%

65 skills spanning six taxonomy categories that cover the Active (wheel-driven), Passive (limb-driven), and Hybrid motion regimes of a wheeled quadruped.

Success Rate and Semantic Alignment. Table. VI compares three LLMs under Direct (zero-shot prompt-to-code) and MimicAgent-based trajectory generation. *No Loop* reports the unit-test success rate of the first attempt. We find that MimicAgent improves over Direct prompting even without iterative improvement, suggesting that the motion planner agent improves generation quality. Further, *Loop i* reports the cumulative unit-test success rate after i rounds of critic-driven repair. However, passing the automated unit tests does not guarantee semantic correctness. We therefore report semantic alignment (SA) to measure the fraction of all skills whose generated trajectory matches its prompt based on human verification. Even with stronger frontier models, the MimicAgent harness considerably improves unit test success rates and semantic alignment over the baseline.

Impact of Unit Tests on Success Rate. We ablate the impact of task-agnostic unit tests on the overall success rate of trajectories generated by MimicAgent in Table VII. To isolate the contribution of different constraints, we define six

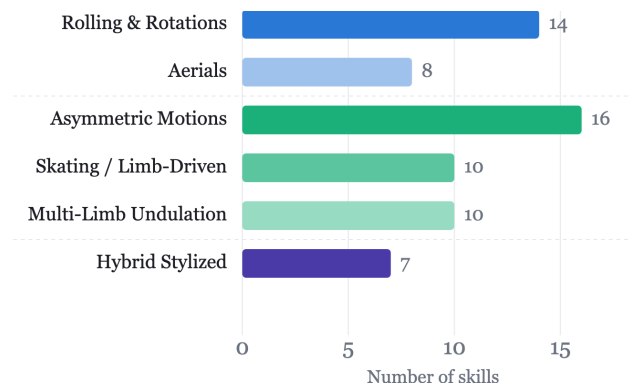


Fig. 3: **LLM Generated Skill Diversity.** MimicAgent generates 65 skills distributed across all six taxonomy categories, covering both 22 Active (wheel-driven), 36 Passive (limb-driven), and 7 Hybrid motion regimes. The balanced spread demonstrates that the pipeline does not collapse onto a narrow subset of the motion space.

TABLE VII: **Unit Test Ablation.** We ablate the impact of each unit test on reference-trajectory and learned-policy quality below, where checkmarks denote the active tests in each configuration. We find that enabling all unit tests performs best, reaching a 92% overall prompt-to-policy alignment.

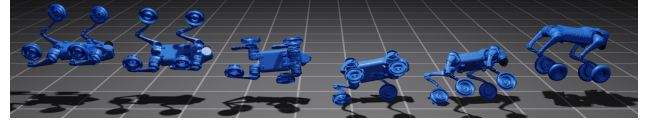
Config	Active Unit Tests					Human Score	
	Height	Joint	Airtime	Ground	LLM	Reference	Policy
C0	×	×	×	×	×	1/5	1/5
C1	✓	×	×	×	×	3/5	2/5
C2	✓	✓	×	×	×	4/5	3/5
C3	✓	✓	✓	×	×	4/5	2/5
C4	✓	✓	✓	✓	×	5/5	4/5
C5	✓	✓	✓	✓	✓	5/5	4/5

RL Policy Semantic Alignment Success Rate: 47/51 (92%)

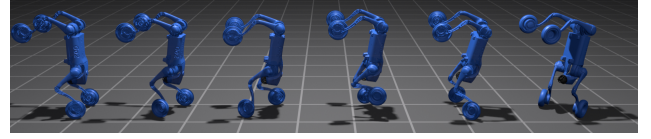
progressive unit test configurations: no unit tests (C0), adding a base height limit (C1), adding joint limit checks (C2), adding a feet airtime penalty (C3), adding ground penetration checks (C4, default used in the main paper), and adding dynamically generated, skill-specific unit tests proposed by an LLM (C5). To evaluate these configurations, we randomly sample five skills from Sonnet 5’s 51 successful skills (79% of 65 skills) passed by MimicAgent Sonnet. For each skill and unit test configuration, we manually validate (e.g. pass / fail) whether the executed skill closely matches the text description. The final success metric for each configuration is the total success rate across the five evaluated skills.

Our results show that progressively adding unit tests improves the overall success rate, with C4 and C5 achieving the highest policy success rate of 80%. The dip observed at C3 can be explained by an interaction between constraints. Specifically, enforcing feet airtime in isolation successfully detects when all feet leave the ground, but compensates by driving the body downward to restore contact. Without a ground penetration constraint, this leads to feet clipping into the terrain. This issue is fully resolved at C4, where the ground penetration constraint is introduced alongside the airtime constraint, demonstrating that these two constraints are complementary and must be applied together to be effective. At C5, LLM-proposed tests introduce skill-specific constraints tailored to the nuances of each motion, resulting in a modest but consistent improvement.

Qualitative Results. We visualize policies trained with MimicAgent reference trajectories in simulation (Figure 4) and real world deployments (Figure 5). We find that MimicAgent can successfully learn dynamic skills in simulation, and highlight that MimicAgent can generate (to the best of our knowledge) never-before-seen skating behavior. Although the reference trajectory does not capture the nuances of skating on underactuated wheels while balancing on two legs, training a policy with DeepMimic *does* capture such subtle dynamics. In Figure 5, we show that walking on two legs is actually transferable to a real robot. We stress that the reference trajectory generated by MimicAgent is not stable, yet the learned RL policy is able to maintain balance while walking.



(a) Backward lateral locomotion with cartwheel-style leg rotations and circular trajectories.



(b) Front legs push-glide-push sequence for skating forward.

Fig. 4: **Qualitative Skill Synthesis Results.** We present representative skills synthesized by MimicAgent. Each sub-figure shows a different skill (with the initial prompt in the subcaption); please see the project page for trajectory rollouts.



(a) Handstand Walk: Walking at 0.4 m/s on challenging outdoor terrain.



(b) Wheelie Roll: Rolling forward at 0.5 m/s on the front wheels while the rear legs are raised and the torso is parallel to ground

Fig. 5: **Sim-to-Real Deployment.** MimicAgent policies transfer directly from simulation to the Go2-W quadruped for both (a) two-legged handstand walking and (b) rolling on the front wheels; please see the project page for rollouts.

Sim-to-Real Metrics. We evaluate 5 skills on the Go2-W over 5 trials each on flat ground of varying roughness (concrete, mat, and polished floor), following the metric conventions of RMA [3] (Table VIII). A trial succeeds only if the robot completes 90% of an episode without an unintended body part touching the ground and matches the high level skill description. We find that Trot and Handstand Walk succeed in all 5 trials at every roughness level, reaching 1.00 normalized time-to-failure (TTF) and distance traveled (DT). Handstand Skate fails a single trial, losing balance while dragging its feet during start-up on rough ground, leaving both TTF and DT at 0.84. Wheelie Roll likewise fails one trial from premature rear-foot contact (80% success). Notably, its DT stays at 0.95 while TTF drops to 0.91; this gap can be attributed to Roll failing on contact violations within an otherwise completed rollout. Lastly, Spin Transition, in which the robot spins in place on all 4 legs and slowly transitions to spinning on its 2 front legs, passes every trial with clean lift-off during the transition phase.

Analysis of Failure Cases. We find that MimicAgent reaches a downstream policy success rate of 92% across 50 generated skills (Table VII). To isolate the effect of reference-trajectory quality on downstream policy performance, we perform a failure-case analysis on the four remaining skills. All four are aerial or in-place rotational skills. Here,

TABLE VIII: **Sim-to-Real Robustness.** We evaluate the success rate and policy rollout quality across five skills.

Skill	SR $\uparrow$	TTF $\uparrow$	DT $\uparrow$
Trot	100% (5/5)	1.00	1.00
Handstand Walk	100% (5/5)	1.00	1.00
Handstand Skate	80% (4/5)	0.84	0.84
Wheelie Roll	80% (4/5)	0.91	0.95
Spin Transition	100% (5/5)	1.00	N/A

two properties coincide. First, tracking objectives of the DeepMimic family are defined primarily on root-relative end-effector positions, so for this skill class a stationary counterfactual that reproduces the reference joint trajectory retains 96.5–99.3% of the perfect-tracking reward. Second, MimicAgent’s trajectory quality degrades on a subset of these failures, producing flight phases that do not follow free-fall motion. Specifically, the robot rises slowly and stays airborne far longer than is physically realizable. We posit that these failures can be attributed to both, since a reference the robot cannot physically track leaves a counterfactual motion that is nearly as rewarding as the intended skill.

V. CONCLUSION

In this paper, we present MimicAgent, an LLM-based agentic pipeline for quadruped trajectory generation. By leveraging coding agents to synthesize coarse yet kinematically feasible reference motions, MimicAgent sidesteps the brittleness and manual effort of reward shaping while avoiding the need for large-scale motion capture datasets. Our results show that these automatically generated trajectories are sufficient for training robust and expressive behaviors across a diverse set of skills. Human evaluations consistently favor policies trained with MimicAgent over Eureka, highlighting our method’s motion quality and language following abilities.

REFERENCES

- [1] G. B. Margolis and P. Agrawal, “Walk these ways: Tuning robot control for generalization with multiplicity of behavior,” in *Conference on Robot Learning*. PMLR, 2023, pp. 22–31.
- [2] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science robotics*.
- [3] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “Rma: Rapid motor adaptation for legged robots,” *arXiv:2107.04034*, 2021.
- [4] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, *et al.*, “Language to rewards for robotic skill synthesis,” *arXiv:2306.08647*, 2023.
- [5] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” *arXiv:2310.12931*, 2023.
- [6] J. Cui, T. Liu, Z. Meng, J. Yu, R. Song, W. Zhang, Y. Zhu, and S. Huang, “Grove: A generalized reward for learning open-vocabulary physical skill,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 15 781–15 790.
- [7] N. Turcato, M. Iovino, A. Synodinos, A. Dalla Libera, R. Carli, and P. Falco, “Towards autonomous reinforcement learning for real-world robotic manipulation with large language models,” *IEEE Robotics and Automation Letters*, 2025.
- [8] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black, “AMASS: Archive of motion capture as surface shapes,” in *International Conference on Computer Vision*, Oct. 2019, pp. 5442–5451.
- [9] A. Allshire, H. Choi, J. Zhang, D. McAllister, A. Zhang, C. M. Kim, T. Darrell, P. Abbeel, J. Malik, and A. Kanazawa, “Visual imitation enables contextual humanoid control,” *arXiv:2505.03729*, 2025.

- [10] S. Xu, H. Y. Ling, Y.-X. Wang, and L.-Y. Gui, “Intermimic: Towards universal whole-body control for physics-based human-object interactions,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 12 266–12 277.
- [11] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi, “Learning human-to-humanoid real-time whole-body teleoperation,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 8944–8951.
- [12] X. B. Peng, A. Kanazawa, J. Malik, P. Abbeel, and S. Levine, “Sfv: Reinforcement learning of physical skills from videos,” *ACM Transactions On Graphics (TOG)*, vol. 37, no. 6, pp. 1–14, 2018.
- [13] X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne, “Deepmimic: Example-guided deep reinforcement learning of physics-based character skills,” *ACM Transactions On Graphics (TOG)*, 2018.
- [14] Z. Zhang, S. Bashkirov, D. Yang, M. Taylor, and X. B. Peng, “Add: Physics-based motion imitation with adversarial differential discriminators,” *arXiv:2505.04961*, 2025.
- [15] G. Bellegarda, M. Shafiee, and A. Ijspeert, “Allgaits: Learning all quadruped gaits and transitions,” in *ICRA*. IEEE, 2025.
- [16] I. Nahrendra, B. Yu, and H. Myung, “Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning,” *arXiv:2301.10602*, 2023.
- [17] L. Smith, J. C. Kew, T. Li, L. Luu, X. B. Peng, S. Ha, J. Tan, and S. Levine, “Learning and adapting agile locomotion skills by transferring experience,” *arXiv:2304.09834*, 2023.
- [18] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, “Feedback mpc for torque-controlled legged robots,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.
- [19] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the mit cheetah 3 through convex model-predictive control,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–9.
- [20] M. H. Raibert, “Trotting, pacing and bounding by a quadruped robot,” *Journal of Biomechanics*, 1990.
- [21] P. Eckert, A. Sprwitz, H. Witte, and A. J. Ijspeert, “Comparing the effect of different spine and leg designs for a small bounding quadruped robot,” in *International Conference on Robotics and Automation*, 2015.
- [22] D. Marhefka, D. Orin, J. Schmiedeler, and K. Waldron, “Intelligent control of quadruped gallops,” *Transactions on Mechatronics*, 2003.
- [23] Y. Li, J. Li, W. Fu, and Y. Wu, “Learning agile bipedal motions on a quadrupedal robot,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 9735–9742.
- [24] R.-Z. Qiu, Y. Song, X. Peng, S. A. Suryadevara, G. Yang, M. Liu, M. Ji, C. Jia, R. Yang, X. Zou, *et al.*, “Wildlma: Long horizon locomotion in the wild,” in *ICRA*. IEEE, 2025.
- [25] K. Zhou, Y. Mu, H. Song, Y. Zeng, P. Wu, H. Gao, and C. Liu, “Adaptive interactive navigation of quadruped robots using large language models,” *arXiv:2503.22942*, 2025.
- [26] P. Jian, X. Wei, Y. Liu, S. A. Moore, M. M. Zavlanos, and B. Chen, “Lapp: Large language model feedback for preference-driven reinforcement learning,” *arXiv:2504.15472*, 2025.
- [27] J. Cui, T. Liu, N. Liu, Y. Yang, Y. Zhu, and S. Huang, “Anyskill: Learning open-vocabulary physical skill for interactive agents,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [28] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, “Translating structured english to robot controllers,” *Advanced Robotics*, 2008.
- [29] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv:2212.06817*, 2022.
- [30] O. Mees, L. Hermann, and W. Burgard, “What matters in language conditioned robotic imitation learning over unstructured data,” *IEEE Robotics and Automation Letters*, 2022.
- [31] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2motion: From natural language instructions to feasible plans,” *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.
- [32] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv:2209.07753*, 2022.
- [33] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” *arXiv:2209.11302*, 2022.
- [34] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *AISTATS*.